

Employee Management System

Project Report — Architecture, Setup, Bug Findings & API Reference

Spring Boot 3.2.0 · Java 17 · MySQL · JWT · Generated June 20, 2026

1. Project Overview

The Employee Management System is a Spring Boot REST API that manages employees and departments, with JWT-based authentication and role-based authorization (ADMIN / USER). It exposes interactive API documentation through Swagger UI and follows a standard layered architecture: controllers handle HTTP requests, services contain business logic, repositories handle persistence via Spring Data JPA, and a global exception handler standardizes error responses.

1.1 Technology Stack

Component	Technology / Version
Language	Java 17
Framework	Spring Boot 3.2.0 (Web, Data JPA, Security, Validation)
Database	MySQL 8 (mysql-connector-j)
ORM	Hibernate / Spring Data JPA, <code>ddl-auto=update</code>
Authentication	JWT via JJWT 0.12.3 (HMAC-SHA signing)
API Docs	springdoc-openapi 2.3.0 (Swagger UI)
Boilerplate	Lombok 1.18.34
Build Tool	Maven

1.2 Package Structure

```
com.example.employee management
├── config           OpenApiConfig
├── controller       AuthController, DashboardController, DepartmentController, EmployeeController
├── dto             AuthRequest, AuthResponse, DashboardStats, DepartmentRequest, EmployeeRequest, EmployeeResponse
├── entity          Department, Employee, EmployeeStatus, Role, User
├── exception       DuplicateResourceException, ResourceNotFoundException, ErrorResponse, GlobalExceptionHandler
├── repository      DepartmentRepository, EmployeeRepository, UserRepository
├── security        JwtService, JwtAuthenticationFilter, SecurityConfig, WebSecurityConfig
└── service         DashboardService, DepartmentService, EmployeeService
```

1.3 Request Flow

Controller → Service → Repository → MySQL. DTOs (`EmployeeRequest` / `EmployeeResponse`, etc.) decouple the external API shape from internal JPA entities for employees; departments currently return entities directly (see Finding 5).

1.4 Authentication & Authorization Flow

1. Client calls `POST /auth/login` with username/password.
2. `AuthenticationManager` (via `DaoAuthenticationProvider`) validates credentials using `UserDetailsService` and BCrypt.
3. `JwtService` issues a signed JWT (subject = username, HMAC signature, configurable expiry).
4. Client sends the token as `Authorization: Bearer <token>` on subsequent calls.
5. `JwtAuthenticationFilter` (a `OncePerRequestFilter`) extracts the username, reloads fresh `UserDetails` (including current role), validates the token, and populates the `SecurityContext`.
6. `WebSecurityConfig` enforces per-route access rules.

1.5 Authorization Rules

Route	Required Role
<code>/auth/**</code> , <code>/swagger-ui/**</code> , <code>/v3/api-docs/**</code>	Public
<code>POST/PUT/DELETE /employees/**</code>	ADMIN
<code>GET /employees/**</code>	Any authenticated user
<code>/departments/**</code> (all methods)	ADMIN
<code>/dashboard/stats</code>	Any authenticated user

2. Setup & Run Instructions

Step 1 — Prerequisites

Install Java 17, Maven 3.6+, and a running MySQL 8 server. Verify with `java -version` and `mvn -version`.

Step 2 — Create the database

Hibernate will create tables automatically (`ddl-auto=update`); you only need the empty schema:

```
CREATE DATABASE employeemanagement;
```

Step 3 — Configure `application.properties`

Update the datasource credentials to match your local MySQL instance:

```
spring.datasource.url=jdbc:mysql://localhost:3306/employeemanagement?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=<your-password>
jwt.secret=<a-secure-32+byte-secret>
jwt.expiration=86400000
```

For anything beyond local testing, move `jwt.secret` to an environment variable instead of committing it to the properties file.

Step 4 — Build

```
mvn clean install
```

Step 5 — Run

```
mvn spring-boot:run
# or
java -jar target/employeemanagement-0.0.1-SNAPSHOT.jar
```

The app starts on port `8080` and Hibernate creates the `users`, `employees`, and `departments` tables on first run.

Step 6 — Register a user and promote to ADMIN

Call `POST /auth/register` with a username/password. The endpoint always assigns role `USER` — there is no API path to create an admin, so promote manually:

```
UPDATE users SET role='ADMIN' WHERE username='your_username';
```

Step 7 — Log in and authorize Swagger

Call `POST /auth/login` to receive a JWT. Open `http://localhost:8080/swagger-ui.html`, click **Authorize**, and paste the raw token (no `Bearer` prefix needed — Swagger adds it).

Step 8 — Exercise the API

Create a department first (`POST /departments`), then create employees against that department's ID (`POST /employees`), then verify with the various GET endpoints and `GET /dashboard/stats`.

3. Bugs Found & Fixes

Five issues were identified during a hands-on walkthrough of the running application, ranging from a critical recursive-serialization defect to smaller logic and search bugs. Each is documented below with its symptom, root cause, and corrected code.

Finding 1 — Department/Employee bidirectional JSON recursion CRITICAL

File: `Department.java`

Symptom: `GET /departments` and `GET /departments/{id}` return a response that grows without bound (department → employees → employee → department → ...), which either fails to parse on the client or can crash the server with a `StackOverflowError`.

Root cause: `Department.employees` and `Employee.department` reference each other. Jackson serializes both sides of the relationship with no cycle-breaking annotation, so it recurses infinitely.

Fix: exclude the back-reference from serialization with `@JsonIgnore`.

```
@OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
@JsonIgnore
private List<Employee> employees = new ArrayList<>();
```

(Import `com.fasterxml.jackson.annotation.JsonIgnore`.) For a richer response that still shows a department's employees, prefer a dedicated DTO with a simplified employee list rather than re-enabling the raw entity reference.

Finding 2 — Missing `@SecurityRequirement` on Department and Dashboard endpoints HIGH

Files: `DepartmentController.java`, `DashboardController.java`

Symptom: Calling these endpoints from Swagger UI returns `403 Forbidden` with an empty body, even after clicking "Authorize" with a valid ADMIN token.

Root cause: `EmployeeController` declares `@SecurityRequirement(name = "bearerAuth")` at the class level, which tells springdoc (and therefore Swagger UI) to attach the bearer token to those requests. `DepartmentController` and `DashboardController` lack this annotation, so Swagger UI sends those requests with no `Authorization` header at all. Spring Security then rejects the anonymous request; since no `httpBasic()` / `formLogin()` entry point is configured, it falls back to a bare 403 instead of 401.

Fix: add the same annotation used on `EmployeeController`.

```
import io.swagger.v3.oas.annotations.security.SecurityRequirement;

@RestController
@RequestMapping("/departments")
@RequiredArgsConstructor
@Tag(name = "Department Management", description = "Department CRUD operations")
@SecurityRequirement(name = "bearerAuth")
public class DepartmentController { ... }
```

```
@RestController
@RequestMapping("/dashboard")
@RequiredArgsConstructor
@Tag(name = "Dashboard", description = "Dashboard statistics API")
@SecurityRequirement(name = "bearerAuth")
public class DashboardController { ... }
```

Finding 3 — `User.enabled` defaults to `false` via Lombok `@Builder` HIGH

File: `User.java`

Symptom: Every newly registered user gets `401 Unauthorized — "Account is disabled"` on their very first login attempt.

Root cause: `private boolean enabled = true;` is a plain field initializer. Lombok's `@Builder` ignores field initializers unless they're marked `@Builder.Default`. Since `AuthController.register()` builds the user without explicitly calling `.enabled(true)`, the builder falls back to Java's primitive default, `false`.

Fix:

```
@Column(nullable = false)
@Builder.Default
private boolean enabled = true;
```

Existing affected rows must also be corrected directly: `UPDATE users SET enabled = true WHERE enabled = false;`

Finding 4 — `searchEmployees()` never searches by email MEDIUM

File: `EmployeeService.java`

Symptom: `GET /employees/search?keyword=...` works for names but returns nothing when the keyword is an email address, despite the endpoint description promising "search by name or email."

Root cause: the method calls `findByFullNameContainingIgnoreCase` twice — once into `byName`, and mistakenly again into what's meant to be `byEmail` — instead of calling an email-matching query the second time.

```
// Before
List<Employee> byName = employeeRepository.findByFullNameContainingIgnoreCase(keyword);
List<Employee> byEmail = employeeRepository.findByFullNameContainingIgnoreCase(keyword); // bug
```

Fix: add an email-matching query to `EmployeeRepository` and call it correctly:

```
// EmployeeRepository.java — add:
List<Employee> findByEmailContainingIgnoreCase(String email);

// EmployeeService.java — fix:
List<Employee> byName = employeeRepository.findByFullNameContainingIgnoreCase(keyword);
List<Employee> byEmail = employeeRepository.findByEmailContainingIgnoreCase(keyword);
```

Finding 5 — Department endpoints expose JPA entities directly

DESIGN RECOMMENDATION

Files: `DepartmentController.java`, `DepartmentService.java`

Observation: unlike `EmployeeController`, which maps to/from `EmployeeRequest` / `EmployeeResponse`, the department endpoints return the raw `Department` entity. This is what made Finding 1 directly visible to API consumers, and more generally risks leaking internal persistence fields (e.g. lazy-loading proxies, timestamps you may not want public) and tightly coupling the API contract to the database schema.

Recommendation: introduce a `DepartmentResponse` DTO (id, name, description, employeeCount, createdAt, updatedAt) and map to it in the service or controller layer, mirroring the existing employee pattern.

4. API Reference

4.1 Authentication

Method	Endpoint	Auth	Description
POST	/auth/register	Public	Register a new user (always assigned role USER)
POST	/auth/login	Public	Authenticate and receive a JWT

Login request body:

```
{ "username": "naitik", "password": "yourpassword" }
```

Login response body:

```
{
  "token": "eyJhbGciOiJIUzUxMiJ9...",
  "username": "naitik",
  "role": "ADMIN"
}
```

4.2 Employees

Method	Endpoint	Auth	Description
POST	/employees	ADMIN	Create a new employee
GET	/employees	Authenticated	List all employees
GET	/employees/{id}	Authenticated	Get one employee by ID
PUT	/employees/{id}	ADMIN	Update an employee
DELETE	/employees/{id}	ADMIN	Delete an employee
GET	/employees/search?keyword=	Authenticated	Search by name (email search currently broken — see Finding 4)

Create employee — request body:

```
{
  "fullName": "John Doe",
  "email": "john.doe@example.com",
  "phoneNumber": "9876543210",
  "departmentId": 1,
  "designation": "Software Engineer",
  "salary": 65000,
  "joiningDate": "2026-01-15",
  "status": "ACTIVE"
}
```

Response body:

```
{
  "id": 1,
  "fullName": "John Doe",
  "email": "john.doe@example.com",
  "phoneNumber": "9876543210",
  "departmentName": "Engineering",
  "designation": "Software Engineer",
  "salary": 65000,
  "joiningDate": "2026-01-15",
  "status": "ACTIVE"
}
```

4.3 Departments

Method	Endpoint	Auth	Description
POST	/departments	ADMIN	Create a department
GET	/departments	ADMIN	List all departments
GET	/departments/{id}	ADMIN	Get one department
PUT	/departments/{id}	ADMIN	Update a department
DELETE	/departments/{id}	ADMIN	Delete a department
GET	/departments/{id}/employee-count	ADMIN	Count of employees in a department

Create department — request body:

```
{ "name": "Engineering", "description": "Handles product development" }
```

4.4 Dashboard

Method	Endpoint	Auth	Description
GET	/dashboard/stats	Authenticated	Aggregate counts for the dashboard

Response body:

```
{
  "totalEmployees": 1,
  "activeEmployees": 1,
  "totalDepartments": 1,
  "recentlyAddedEmployeesCount": 1
}
```

4.5 Error Response Shape

Most error responses (404, 409, 401) follow a consistent shape produced by `GlobalExceptionHandler`:

```
{
  "timestamp": "2026-06-20T23:13:46.564",
  "status": 401,
  "error": "Unauthorized",
  "message": "Account is disabled",
  "path": "/auth/login"
}
```

Validation errors (400, from `@Valid` failures) instead return a flat field-to-message map, e.g. `{ "email": "Invalid email format" }`, rather than the `ErrorResponse` shape above.